PRACTICAL - 10

Implementation of Kruskal's Algorithm

Aim of the Practical:

To implement Kruskal's algorithm using Disjoint Set Union (DSU) and analyze the cost of sorting and cycle detection.

Program:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct Edge { int u, v, w; };
struct DSU {
    vector<int> p, rankv;
    DSU(int n): p(n), rankv(n,0) { for(int i=0;i<n;i++) p[i]=i; }
    int find(int x) { return p[x]==x ? x : p[x]=find(p[x]); }
    bool unite(int a,int b) {
        a=find(a); b=find(b);
        if(a==b) return false;
        if(rankv[a]<rankv[b]) swap(a,b);
        p[b]=a;
        if(rankv[a]==rankv[b]) rankv[a]++;
        return true;
    }
};
int main() {
    int V = 4;
    vector<Edge> edges = {
        {0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}
    };
    sort(edges.begin(), edges.end(),
         [](Edge a, Edge b){ return a.w < b.w; });
    DSU dsu(V);
    int cost = 0;
    for (auto e : edges) {
        if (dsu.unite(e.u, e.v)) {
            cout << e.u << " - " << e.v << " : " << e.w << endl;
            cost += e.w;
        }
    }
    cout << "MST cost = " << cost;
}
```

Output:

```
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
MST cost = 19
```

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices and E the number of edges.

First, all E edges are sorted by weight.

$$\text{Sorting cost} = \Theta(E \log E)$$

Then each edge is considered at most once. For each edge, DSU performs find/unite operations.

With path compression and union by rank, the amortized cost of a DSU operation is $O(\alpha(V))$, where α is the inverse Ackermann function.

$$\text{DSU processing cost} = O(E\ \alpha(V))$$

$$T(V,E) = \Theta(E \log E) + O(E\ \alpha(V))$$

Name: SHAIK ASHWAK BASHA **Enrollment No: 92460118522**

Since sorting dominates:

$$T(V,E) = \Theta(E \log E)$$

For a simple graph $E \leq V^2$, $\log E = O(\log V)$, so this is also commonly written $O(E \log V)$.

Final Time Analysis:

- Time Complexity: $\Theta(E \log E)$, dominated by sorting.
- DSU auxiliary storage: $\Theta(V)$.
- Edge-list storage: $\Theta(E)$.

Result:

Kruskal's algorithm successfully constructed the MST and avoided cycles using DSU.